Module 5 - NODE JS

SUNITA D RATHOD

Introduction

Node.js is a cross-platform environment and library for running JavaScript applications which is used to create networking and server-side applications.

Node.js is a compelling JavaScript-based platform that's built on Google Chrome's JavaScript V8 Engine.

It is used to develop I/O intensive web applications, such as video streaming sites, single-page applications, online chat applications, and other web apps.

Node js

Node.js is also provide a rich library of various JavaScript modules to simplify the development of web applications.

Node.js= Runtime Environment + JavaScript Library

It is open source and free to use.

Features of Node.js

- Open source
- Simple & Fast
- Asynchronous
- High Scalability
- Single Threaded
- No Buffering
- Cross platform
- license

Advantages of Node.js

- Easy to Scale
- Real time web applications
- Faster suite
- Easy to learn
- Single programming languages
- Caching
- Data streaming
- Hosting
- Support of large and Active community

Applications of Node.js

- Real time chats
- Complex Single page Applications
- REal time Collaboration tools
- Live streaming application
- JSON APIs based applications

Major brands that uses node.js

Netflix, linkedIn, uber, paypal, eBay, NASA etc,

Environment Setup

If you want to set up your environment for Node.js, you need to have the following two software on your computer,

(a)Text Editor : Examples of text editors include Windows Notepad, OS Edit command, Brief, Epsilon, EMACS, and vim or vi.

(b) the Node.js binary installables : Node.js distribution comes as a binary installable for SunOS, Linux, Mac OS X, and Windows operating systems with the 32-bit (386) and 64-bit (amd64) x86 processor architectures

- Download the latest version of Node.js installable archive file from
- https://nodejs.org/en/download/

Node.js Data Types

Primitive data types

- String
- Number
- Boolean
- Null
- Undefined

Non-Primitive data types are :

- Object
- Date
- Array

Variables

Syntax:

```
var varName = value ;
```

Operators

Operator Types	Operators
Arithmetic	+, - , / , * , % , ++ , - -
Assignment	=, += , -=, *= , /= ,%=
Conditional	? :
Comparison	<, > , <=, >=, ==, !=, ===, !==
Logical	&&, || , !
Bitwise	& , | , ^ , << , >>

Functions

Syntax

```
Function display_Name( firstName, lastName)
{
alert(“ Hello” + firstName + “” + lastName);
}

display_Name(“ Amar”, “ Bobby”);
```

File System

fs module is a build-in module in node js used to **access filesystem**.

fs module has methods to *read*, *write*, *append*, *rename*, and *delete* data from a file stored in **File System**.

var fs = require('fs') ;

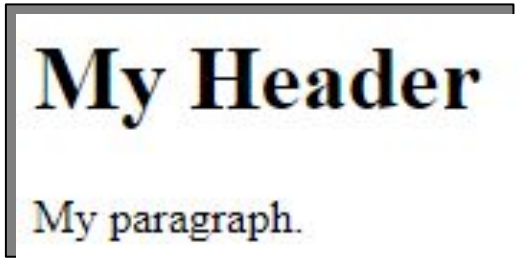
demofile1.html

```
<html>
<body>
<h1>My Header</h1>
<p>My paragraph.</p>
</body>
</html>
```

Read File

The `fs.readFile()` method is used to read files on your computer.

```
var http = require('http');
var fs = require('fs');
http.createServer(function (req, res) {
  fs.readFile('demofile1.html', function(err, data) {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    return res.end();
  });
}).listen(8080);
```



Create Files

The File System module has methods for creating new files:

- fs.appendFile()
- fs.open()
- fs.writeFile()

```
var fs = require('fs');

fs.open('mynewfile2.txt', 'w', function (err, file) {
  if (err) throw err;
  console.log('Saved!');
});
```

Update Files

The File System module has methods for updating files:

- fs.appendFile()
- fs.writeFile()

```
var fs = require('fs');

fs.appendFile('mynewfile1.txt', ' This is my text.', function (err) {
  if (err) throw err;
  console.log('Updated!');
});
```

```
var fs = require('fs');

fs.writeFile('mynewfile3.txt', 'This is my text', function (err) {
  if (err) throw err;
  console.log('Replaced!');
});
```

Delete Files

To delete a file with the File System module, use the fs.unlink() method.

```
var fs = require('fs');

fs.unlink('mynewfile2.txt', function (err) {
  if (err) throw err;
  console.log('File deleted!');
});
```

Rename Files

To rename a file with the File System module, use the fs.rename() method.

```
var fs = require('fs');

fs.rename('mynewfile1.txt', 'myrenamedfile.txt', function (err) {
  if (err) throw err;
  console.log('File Renamed!');
});
```

NPM (NODE PACKAGE MANAGER)

NPM is **Node Package Manager**, the default package manager of Node JS.

NPM always comes with Node JS.

To check whether **NPM** is installed with node in your system, type npm in terminal.

To check **npm version**, type

npm --version.

Node.js Modules

Consider modules to be the same as JavaScript libraries.

A set of functions you want to include in your application.

Built-in Modules

Node.js has a set of built-in modules which you can use without any further installation.

Creating node.js application

A Node.js application consists of the following three important components −

• Import required modules − We use the require directive to load Node.js modules.

• Create server − A server which will listen to client's requests similar to Apache HTTP Server.

• Read request and return response − The server created in an earlier step will read the HTTP request made by the client which can be a browser or a console and return the response.

Include Modules

To include a module, use the **require()** function with the name of the module:

```
var http = require('http');
```

Now your application has access to the HTTP module, and is able to create a server:

```
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

Create Your Own Modules

You can create your own modules, and easily include them in your applications.

The following example creates a module that returns a date and time object:

Create a module that returns the current date and time:

```
exports.myDateTime = function () {
  return Date();
};
```

Save the code above in a file called "myfirstmodule.js"

Include Your Own Module

```
var http = require('http');
var dt = require('./myfirstmodule');

http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write("The date and time are currently: " + dt.myDateTime());
  res.end();
}).listen(8080);
```

The date and time are currently Wed Sep 18 2024 14:22:26 GMT+0530 (India Standard Time)

Asynchronous Model

Node is non-blocking which means it can take many requests at once.

In Node programming model almost everything is done asynchronously but many of the functions in Node.js core have both synchronous and asynchronous versions.

Under most situation, we should use the asynchronous functions to get the advantages of Node.js.

Here is an example comparing synchronous and asynchronous model using file_system.readFile

Example : Synchronously reads the file

Here we have used fs.readFileSync() which is the synchronous version of fs.readFile(). It also returns the contents of the filename.

```
var fs = require('fs');
var content = fs.readFileSync("readme.txt", "utf8");
console.log(content);
console.log('Reading file...');
```

Output:

```
E:\nodejs>node file-read-syn.js
Node.js tutorial from w3resouce.
Reading file...

E:\nodejs>
```

Example : Asynchronously reads the file

```
var fs = require('fs');
fs.readFile("readme.txt", "utf8", function(err, content) {
if (err) {
      return console.log(err);
               }
  console.log(content);
      });
console.log('Reading file...');
```

Output :

```
E:\nodejs>node file-read-asyn.js
Reading file...
Node.js tutorial from w3resouce.

E:\nodejs>
```

CALL BACKS

A callback function is a function that is passed to another function as a parameter, and the callback function is called (executed) inside the second function.

Callbacks functions execute asynchronously, instead of reading top to bottom procedurally.

CALL BACKS

For example, we are trying to download a large file and we don't want that server blocks all other request to complete the download job, rather we want to send this download job in the background and to call a function when it's finished.

Therefore server should follow the second rule and can accept other request while it is doing something in the background and makes other code non-blocking.

CALL BACKS

In a synchronous program, you can write codes in the following way :

```
1  var content;
2  function readingfile() {
3  var fs = require('fs');
4  content = fs.readFileSync("readme.txt", "utf8");
5  return content;
6  }
7  readingfile();
8  console.log(content);
```

CALL BACKS

Node.js uses callbacks, being an asynchronous platform, it does not wait around like database query, file I/O to complete.

```
1  var fs = require('fs');
2  var fcontent ;
3  function readingfile(callback){
4  fs.readFile("readme.txt", "utf8", function(err, content) {
5  fcontent=content;
6   if (err)
7   {
8   return console.error(err.stack);
9    }
10 callback(content);
11 })
12 };
13 function mycontent() {
14   console.log(fcontent);
15 }
16 readingfile(mycontent);
17 console.log('Reading files....');
```

EVENT LOOP

Node is a single-threaded event-driven platform that is capable of running non-blocking, asynchronous programming. These functionalities of Node make it memory efficient.

What is the Event Loop?

The **event loop** allows Node to perform non-blocking I/O operations despite the fact that JavaScript is single-threaded. It is done by assigning operations to the operating system whenever and wherever possible.

Why Event Loop is important?

Most operating systems are multi-threaded and hence can handle multiple operations executing in the background.

When one of these operations is completed, the kernel tells Node.js, and the respective callback assigned to that operation is added to the event queue which will eventually be executed.

Features of Event Loop:

- An event loop is an endless loop, which waits for tasks, executes them, and then sleeps until it receives more tasks.
- The event loop executes tasks from the event queue only when the call stack is empty i.e. there is no ongoing task.
- The event loop allows us to use callbacks and promises.
- The event loop executes the tasks starting from the oldest first.

Example

```
console.log("This is the first statement");
setTimeout(function(){
        console.log("This is the second statement");
}, 1000);
console.log("This is the third statement");
```

```
This is the first statement
This is the third statement
This is the second statement
```

Working of the event loop

When Node.js starts, it initializes the event loop, processes the provided input script which may make async API calls, schedules timers, then begins processing the event loop.

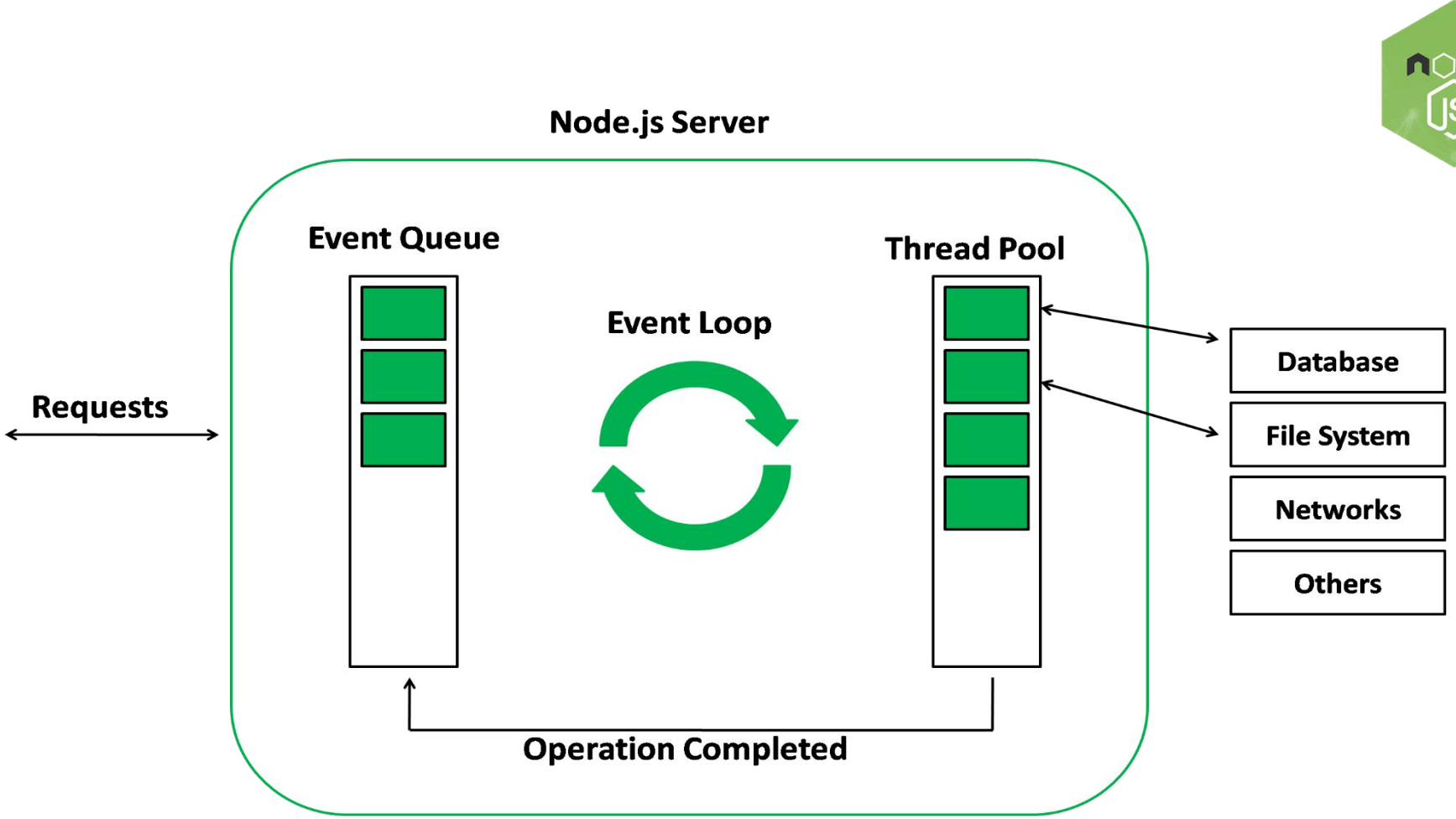
When using Node.js, a special library module called libuv is used to perform async operations. This library is also used, together with the back logic of Node, to manage a special thread pool called the libuv thread pool.

Working of the event loop

This thread pool is composed of four threads used to delegate operations that are too heavy for the event loop. I/O operations, Opening and closing connections, setTimeouts are examples of such operations.

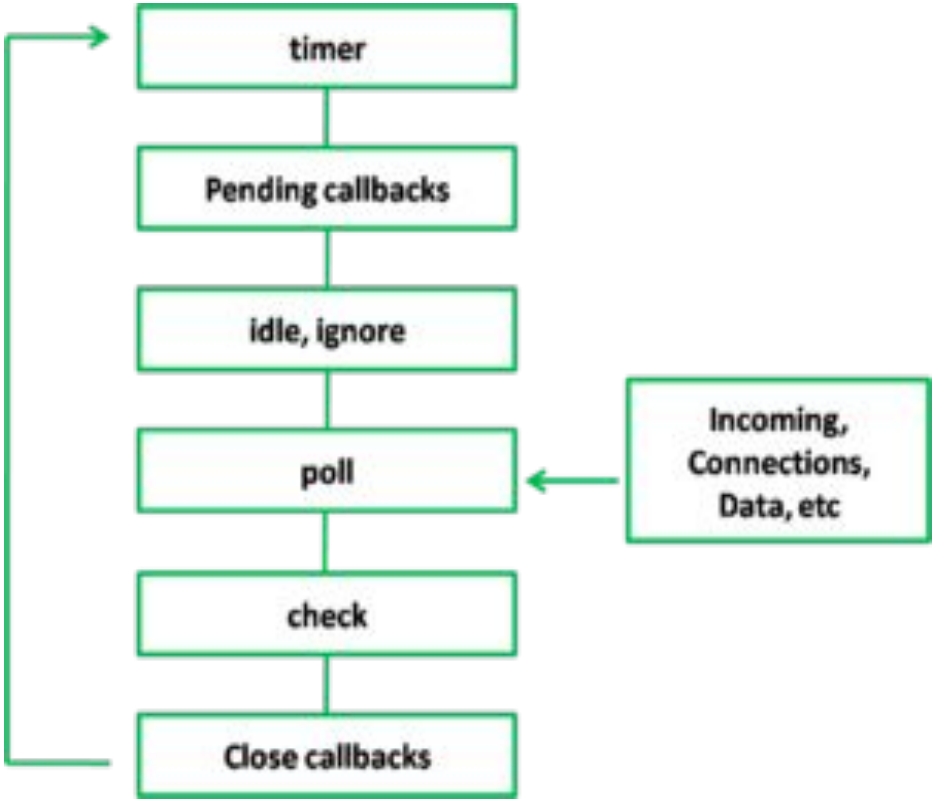
When the thread pool completes a task, a callback function is called which handles the error(if any) or does some other operation. This callback function is sent to the event queue.

When the call stack is empty, the event goes through the event queue and sends the callback to the call stack.


Node.js Server
Event Queue
Thread Pool
Event Loop
Requests
Database
File System
Networks
Others
Operation Completed

Phases of the event loop

The event loop in Node.js consists of several phases, each of which performs a specific task. These phases include:



Phases of the event loop

1. Timers: This phase processes timers that have been set using setTimeout() and setInterval().

2. Pending Callbacks: This phase processes any callbacks that have been added to the message queue by asynchronous functions.

3. Idle, Prepare: used internally only

4. Poll: This phase is used to check for new I/O events and process any that have been detected.

5. Check This phase processes any setImmediate() callbacks that have been added to the message queue.

6. Close Callbacks: This phase processes any callbacks that have been added to the message queue by the close event of a socket.

REPL

The term REPL stands for Read Eval Print and Loop.

It specifies a computer environment like a window console or a Unix/Linux shell where you can enter the commands and the system responds with an output in an interactive mode.

It is useful in writing and debugging the codes.

REPL Environment

The Node.js or node come bundled with REPL environment. Each part of the REPL environment has a specific work.

- Read: It reads user's input, parse the input into JavaScript data-structure and stores in memory.
- Eval: It takes and evaluates the data structure.
- Print: It prints the result.
- Loop: It loops the above command until user press ctrl-c twice.

Node.js repl

• Starting REPL

REPL can be started by simply running node on shell/console without any arguments as follows.

$ node

You will see the REPL command prompt > where you can type any node.js command

$ node

> 1 + 3

4

Use Variables

You can make use variables to store values and print later like any conventional script.

If var keyword is not used, then the value is stored in the variable and printed. Whereas if var keyword is used, then the value is stored but not printed. You can print variables using console.log()

Example

```
$ node
> x = 10
10
> var y = 10
undefined
> x + y
20
```

```
> console.log("Hello World")
Hello World
undefined
```

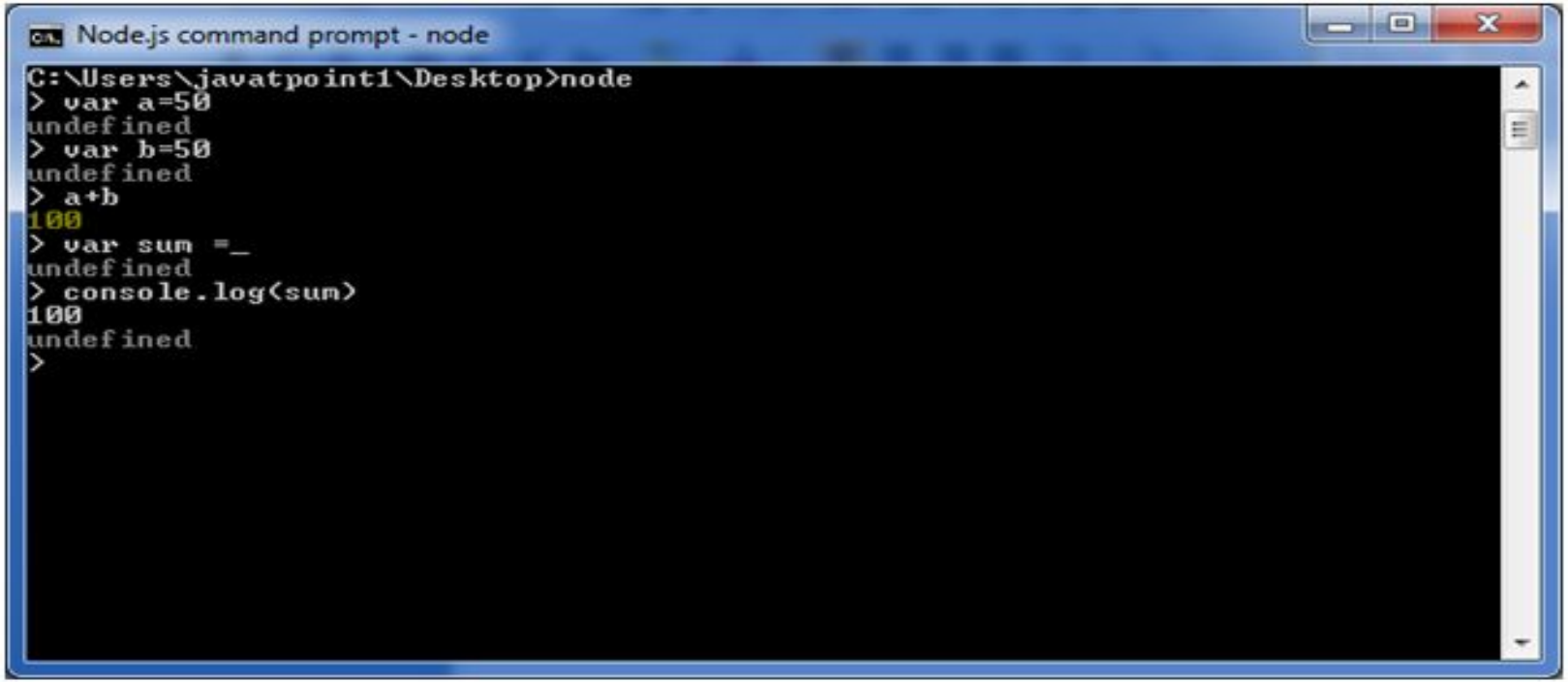
Multiline Expression

Node REPL supports multiline expression similar to JavaScript. Let's check the following do-while loop in action:

```
> node
> var x = 0
undefined
> do {
... x++;
... console.log("x: " + x);
... } while ( x < 5 );
x: 1
x: 2
x: 3
x: 4
x: 5
undefined
>
```

Node.js Underscore Variable

You can also use underscore _ to get the last result.

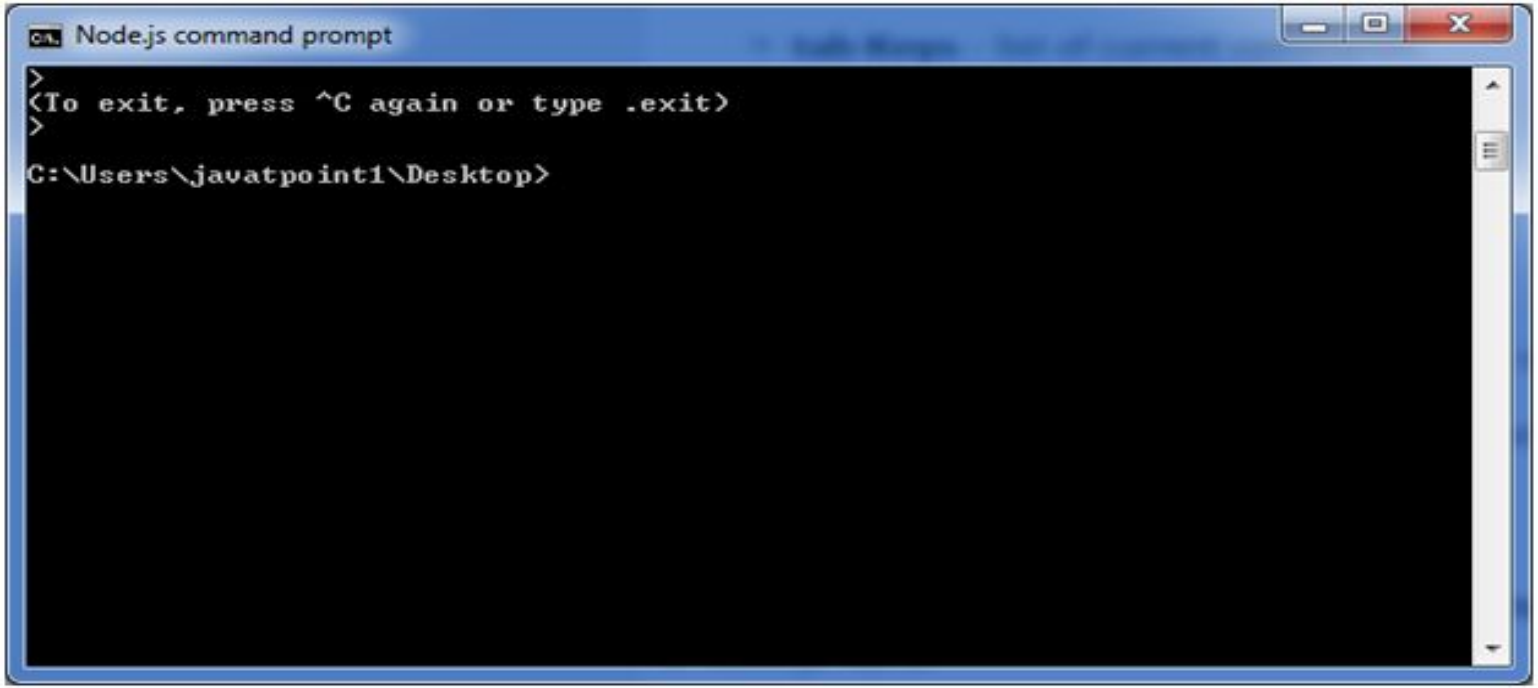


Node.js REPL Commands

Commands	Description
ctrl + c	It is used to terminate the current command.
ctrl + c twice	It terminates the node repl.
ctrl + d	It terminates the node repl.
up/down keys	It is used to see command history and modify previous commands.
tab keys	It specifies the list of current command.
.help	It specifies the list of all commands.
.break	It is used to exit from multi-line expressions.
.clear	It is used to exit from multi-line expressions
.save filename	It saves current node repl session to a file.
.load filename	It is used to load file content in current node repl session.

Stopping REPL

Use ctrl + c command twice to come out of Node.js REPL.



node.js eventemitter

The EventEmitter class in Node.js is a core module that provides a way to handle asynchronous events.

It objects to emit events and other objects to listen and respond to those events.

Event Emitter in Node

Node.js uses events module to create and handle custom events. The EventEmitter class can be used to create and handle custom events module.

Importing EventEmitter

The syntax to Import the events module are given below:

Syntax:

```
const EventEmitter = require('events');
```

All EventEmitters emit the event **newListener** when new listeners are added and **removeListener** when existing listeners are removed. It also provide one more option:

boolean captureRejections

```
Default Value: false
It automatically captures rejections.
```

Listening events

Before emits any event, it must register functions(callbacks) to listen to the events.

Syntax:

```
eventEmitter.addListener(event, listener)
eventEmitter.on(event, listener)
```

eventEmitter.on(event, listener) and **eventEmitter.addListener(event, listener)** are pretty much similar. It adds the listener at the end of the listener's array for the specified event. Multiple calls to the same event and listener will add the listener multiple times and correspondingly fire multiple times. Both functions return emitter, so calls can be chained.

Emitting events

Every event is named event in nodejs. We can trigger an event by emit(event, [arg1], [arg2], […]) function. We can pass an arbitrary set of arguments to the listener functions.

Syntax:

```
eventEmitter.emit(event, [arg1], [arg2], [...])
```

example

This example demonstrates how to create an EventEmitter instance in Node.js, register a listener for a custom event, and emit that event with a message.

```
// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
let eventEmitter = new EventEmitter();

// Registering to myEvent
eventEmitter.on('myEvent', (msg) => {
  console.log(msg);
});

// Triggering myEvent
eventEmitter.emit('myEvent', "First event");
```

Output:

```
First event
```

Removing Listener

The **eventEmitter.removeListener()** takes two argument event and listener, and removes that listener from the listeners array that is subscribed to that event. While **eventEmitter.removeAllListeners()** removes all the listener from the array which are subscribed to the mentioned event.

Syntax:

eventEmitter.removeListener(event, listener)
eventEmitter.removeAllListeners([event])

example

This example demonstrates how to register multiple listeners for an event using Node.js EventEmitter, remove a specific listener, and remove all listeners, showing the flexibility in managing event handling

```
// Importing events
const EventEmitter = require('events');
// Initializing event emitter instances
let eventEmitter = new EventEmitter();
 let geek1= (msg) => {
   console.log("Message from geek1: " + msg);
};
let geek2 = (msg) => {
   console.log("Message from geek2: " + msg);
};
// Registering geek1 and geek2
eventEmitter.on('myEvent', geek1);
eventEmitter.on('myEvent', geek1);
eventEmitter.on('myEvent', geek2);
 // Removing listener geek1 that was
      // registered on the line 13
      eventEmitter.removeListener('myEvent',
      geek1);

      // Triggering myEvent
      eventEmitter.emit('myEvent', "Event
      occurred");
      // Removing all the listeners to myEvent
      eventEmitter.removeAllListeners('myEvent');

      // Triggering myEvent
      eventEmitter.emit('myEvent', "Event
      occurred");
```

Output:

```
Message from geek1: Event occurred
Message from geek2: Event occurred
```

We registered two times geek1 and one time geek2. For calling eventEmitter.removeListener('myEvent', geek1) one instance of geek1 will be removed. Finally, removing all listener by using removeAllListeners() method that will remove all listeners to myEvent.

Special Events

All EventEmitter instances emit the event **‘newListener’** when new listeners are added and **‘removeListener’** existing listeners are removed.

Event: ‘newListener’ The EventEmitter instance will emit its own ‘newListener’ event before a listener is added to its internal array of listeners. Listeners registered for the ‘newListener’ event will be passed to the event name and reference to the listener being added. The event ‘newListener’ is triggered before adding the listener to the array.

eventEmitter.once('newListener', listener)
eventEmitter.on('newListener', listener)

Event: ‘removeListener’ The **‘removeListener’** event is emitted after a listener is removed.
eventEmitter.once(‘removeListener’, listener)
eventEmitter.on('removeListener’, listener)

Event: ‘error’ When an error occurs within an EventEmitter instance, the typical action is for an **‘error’** event to be emitted. If an EventEmitter does not have at least one listener registered for the ‘error’ event, and an ‘error’ event is emitted, the error is thrown, a stack trace is printed, and the Node.js process exits.

eventEmitter.on('error', listener)

Example: This example demonstrates how to handle special events like newListener and removeListener in Node.js' EventEmitter, alongside registering and removing custom listeners, and handling errors effectively.

```
// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
let eventEmitter = new EventEmitter();

// Register to error
eventEmitter.on('error', (err) => {
    console.error('Attention! There was an error');
});

// Register to newListener
eventEmitter.on( 'newListener', (event, listener) => {
    console.log(`The listener is added to ${event}`);
});
```

```
// Register to removeListener
eventEmitter.on( 'removeListener', (event, listener) => {
    console.log(`The listener is removed from  ${event}`);
});

// Declaring listener geek1 to myEvent1
let geek1   = (msg) => {
    console.log("Message from geek1: "  + msg);
};

// Declaring listener geek2 to myEvent2
let geek2 = (msg) => {
    console.log("Message from geek2: "  + msg);
};
```

```
// Listening to myEvent with geek1 and geek2
eventEmitter.on('myEvent', geek1);
eventEmitter.on('myEvent', geek2);

// Removing listener
eventEmitter.off('myEvent', geek1);

// Triggering myEvent
eventEmitter.emit('myEvent', 'Event occurred');

// Triggering error
eventEmitter.emit('error', new Error('Attention!'));
```

Output:

```
The listener is added to removeListener

The listener is added to myEvent

The listener is added to myEvent

The listener is removed from myEvent

Message from geek2: Event occurred

Attention! There was an error
```

Node.js Buffers

Node.js provides Buffer class to store raw data similar to an array of integers but corresponds to a raw memory allocation outside the V8 heap.

Buffer class is used because pure JavaScript is not nice to binary data.

So, when dealing with TCP streams or the file system, it's necessary to handle octet streams.

Buffer class is a global class.

It can be accessed in application without importing buffer module.

Node.js Creating Buffers

There are many ways to construct a Node buffer. Following are the three mostly used methods:

1. Create an uninitiated buffer: Following is the syntax of creating an uninitiated buffer of 10 octets:

```
var buf = new Buffer(10);
```

2. Create a buffer from array: Following is the syntax to create a Buffer from a given array:

```
var buf = new Buffer([10, 20, 30, 40, 50]);
```

Node.js Creating Buffers

3. Create a buffer from string: Following is the syntax to create a Buffer from a given string and optionally encoding type:

```
var buf = new Buffer("Simply Easy Learning", "utf-8");
```

Node.js Writing to buffers

Following is the method to write into a Node buffer:

Syntax:

buf.write(string[, offset][, length][, encoding])

Parameter explanation:

string: It specifies the string data to be written to buffer.

offset: It specifies the index of the buffer to start writing at. Its default value is 0.

length: It specifies the number of bytes to write. Defaults to buffer.length

encoding: Encoding to use. 'utf8' is the default encoding.

Return values from writing buffers:

This method is used to return number of octets written. In the case of space shortage for buffer to fit the entire string, it will write a part of the string.

example:

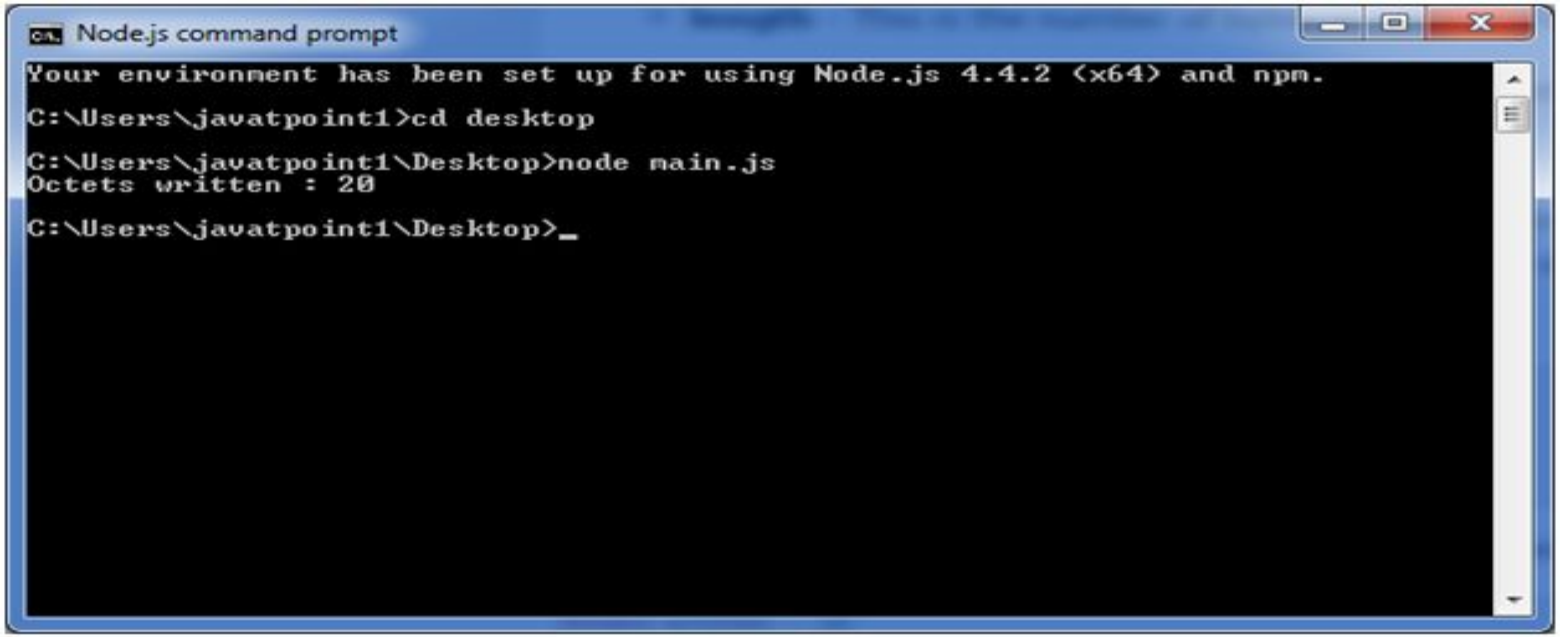
Create a JavaScript file named "main.js" having the following code:

File: main.js

```
buf = new Buffer(256);
len = buf.write("Simply Easy Learning");
console.log("Octets written : "+  len);
```

Open the Node.js command prompt and execute the following code:

```
node main.js
```



Node.js Reading from buffers

Following is the method to read data from a Node buffer.

Syntax:

buf.toString([encoding][, start][, end])

Parameter explanation:

encoding: It specifies encoding to use. 'utf8' is the default encoding
start: It specifies beginning index to start reading, defaults to 0.
end: It specifies end index to end reading, defaults is complete buffer.

Return values reading from buffers: This method decodes and returns a string from buffer data encoded using the specified character set encoding.

Example

File: main.js

```
buf = new Buffer(26);
for (var i = 0 ; i < 26 ; i++) {
  buf[i] = i + 97;
}
console.log( buf.toString('ascii'));       // outputs:
abcdefghijklmnopqrstuvwxyz
console.log( buf.toString('ascii',0,5));   // outputs: abcde
console.log( buf.toString('utf8',0,5));    // outputs: abcde
console.log( buf.toString(undefined,0,5)); // encoding defaults to
'utf8', outputs abcde
```

Open Node.js command prompt and execute the following code:

```
node main.js
```

OUTPUT


Node.js command prompt
Your environment has been set up for using Node.js 4.4.2 (x64) and npm.
C:\Users\javatpoint1>cd desktop
C:\Users\javatpoint1\Desktop>node main.js
abcdefghijklmnopqrstuvwxyz
abcde
abcde
abcde
C:\Users\javatpoint1\Desktop>

Node.js Streams

Streams are the objects that facilitate you to read data from a source and write data to a destination.

There are four types of streams in Node.js:

1. **Readable:** This stream is used for read operations.
2. **Writable:** This stream is used for write operations.
3. **Duplex:** This stream can be used for both read and write operations.
4. **Transform:** It is type of duplex stream where the output is computed according to input.

Node.js Streams

Each type of stream is an Event emitter instance and throws several events at different times. Following are some commonly used events:

- **Data:**This event is fired when there is data available to read.
- **End:**This event is fired when there is no more data available to read.
- **Error:** This event is fired when there is any error receiving or writing data.
- **Finish:**This event is fired when all data has been flushed to underlying system.

Node.js Reading from stream

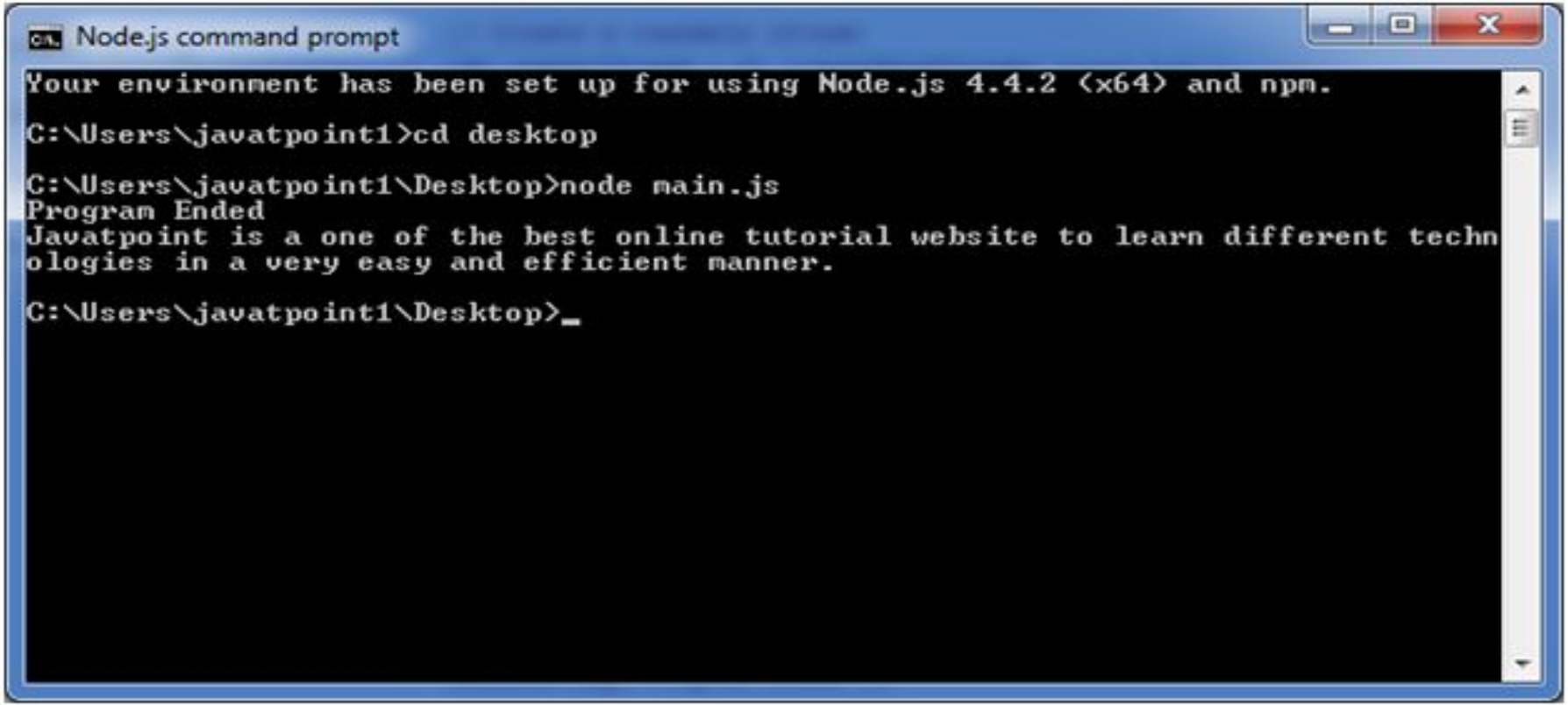
Create a text file named input.txt having the following content:

Javatpoint is a one of the best online tutorial website to learn different technologies in a very easy and efficient manner.

File: main.js

```
var fs = require("fs");
var data = '';
// Create a readable stream
var readerStream =
fs.createReadStream('input.txt');
// Set the encoding to be utf8.
readerStream.setEncoding('UTF8');
// Handle stream events --> data,
end, and error
readerStream.on('data',
function(chunk) {
   data += chunk;
});
readerStream.on('end',function()
{
   console.log(data);
});
readerStream.on('error',
function(err){
   console.log(err.stack);
});
console.log("Program Ended");
```

output


Node.js command prompt
Your environment has been set up for using Node.js 4.4.2 (x64) and npm.
C:\Users\javatpoint1>cd desktop
C:\Users\javatpoint1\Desktop>node main.js
Program Ended
Javatpoint is a one of the best online tutorial website to learn different techn
ologies in a very easy and efficient manner.
C:\Users\javatpoint1\Desktop>

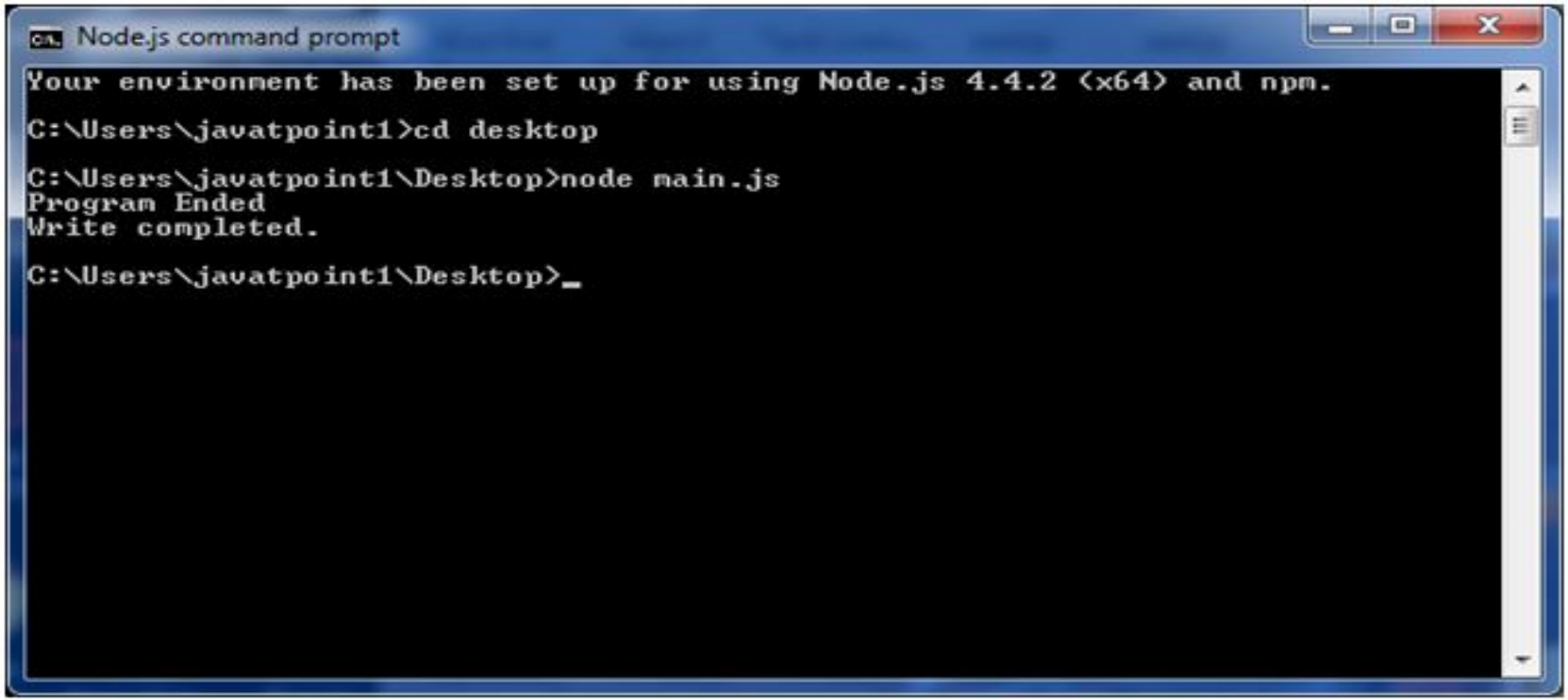
Node.js Writing to stream

Create a JavaScript file named main.js having the following code:

```
var fs = require("fs");
var data = 'A Solution of all Technology';
// Create a writable stream
var writerStream =
fs.createWriteStream('output.txt');
// Write the data to stream with encoding to be
utf8
writerStream.write(data,'UTF8');
// Mark the end of file
writerStream.end();
```

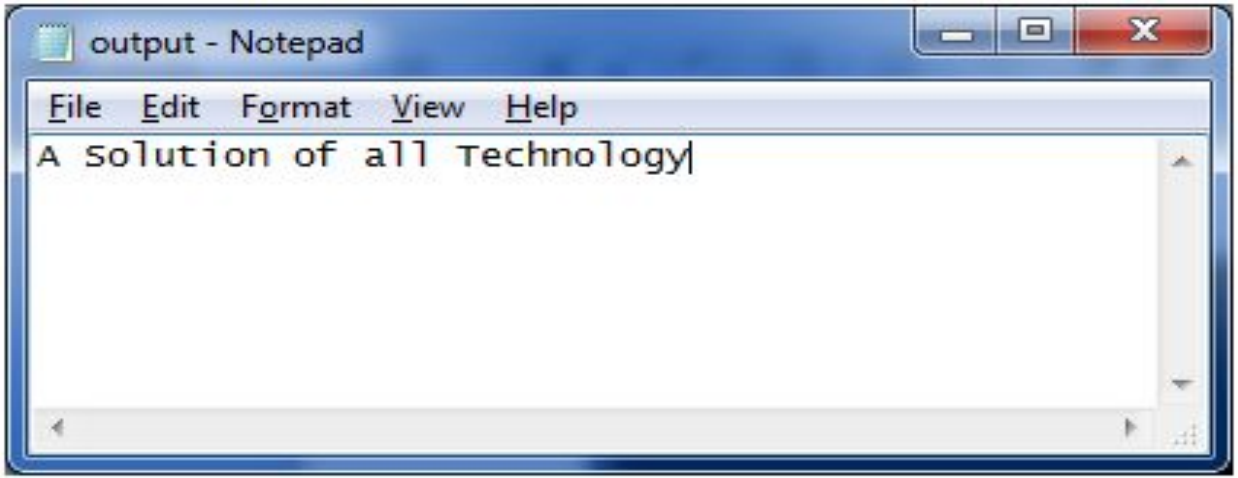
```
// Handle stream events -->
finish, and error
writerStream.on('finish',
function() {
   console.log("Write
completed.");
});
writerStream.on('error',
function(err){
  console.log(err.stack);
});
console.log("Program Ended");
```

output


Node.js command prompt
Your environment has been set up for using Node.js 4.4.2 (x64) and npm.
C:\Users\javatpoint1>cd desktop
C:\Users\javatpoint1\Desktop>node main.js
Program Ended
Write completed.
C:\Users\javatpoint1\Desktop>

Now, you can see that a text file named "output.txt" is created where you had saved "input.txt" and "main.js" file. In my case, it is on desktop.

Open the "output.txt" and you will see the following content.



Node.js Piping Streams

Piping is a mechanism where output of one stream is used as input to another stream.

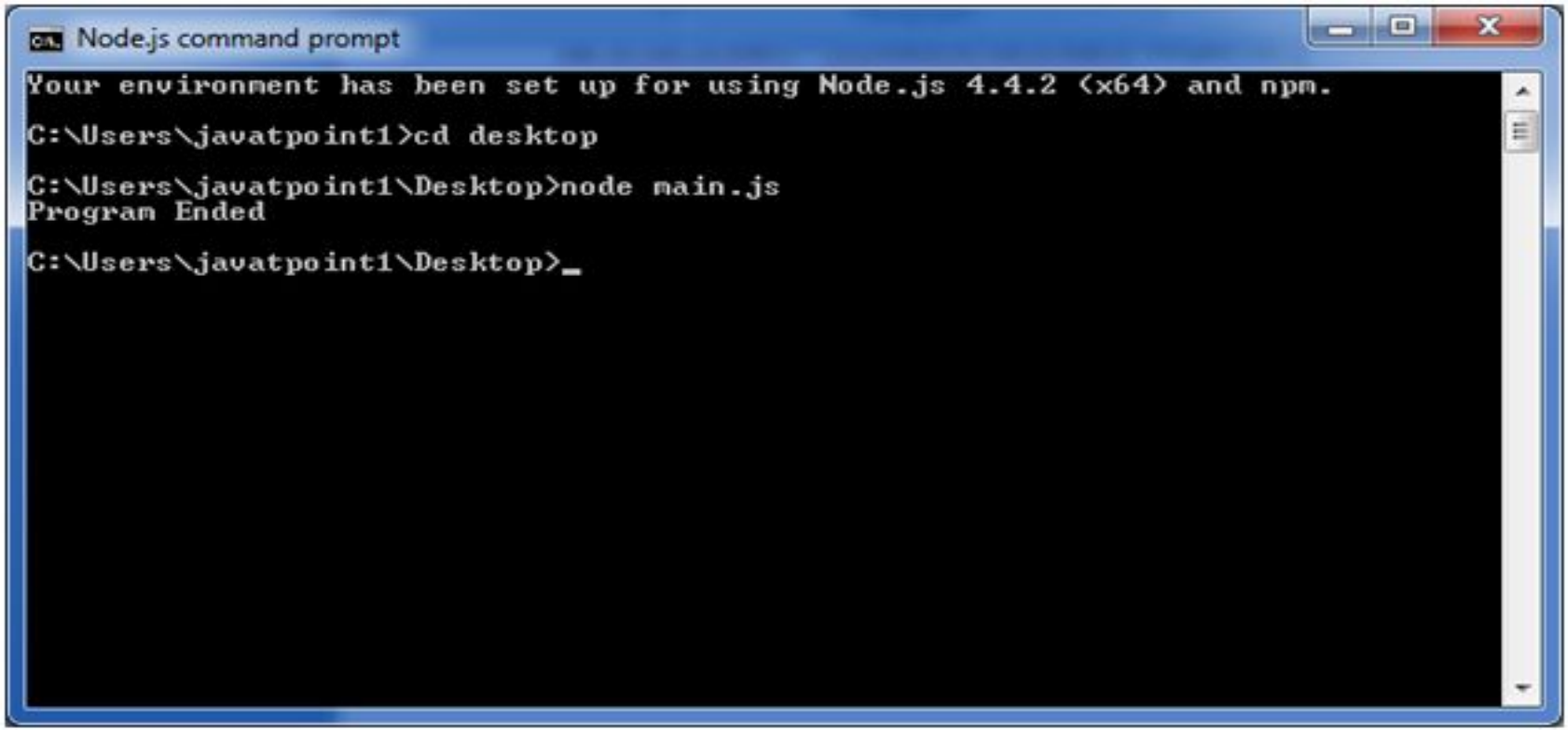
There is no limit on piping operation.

Let's take a piping example for reading from one file and writing it to another file.

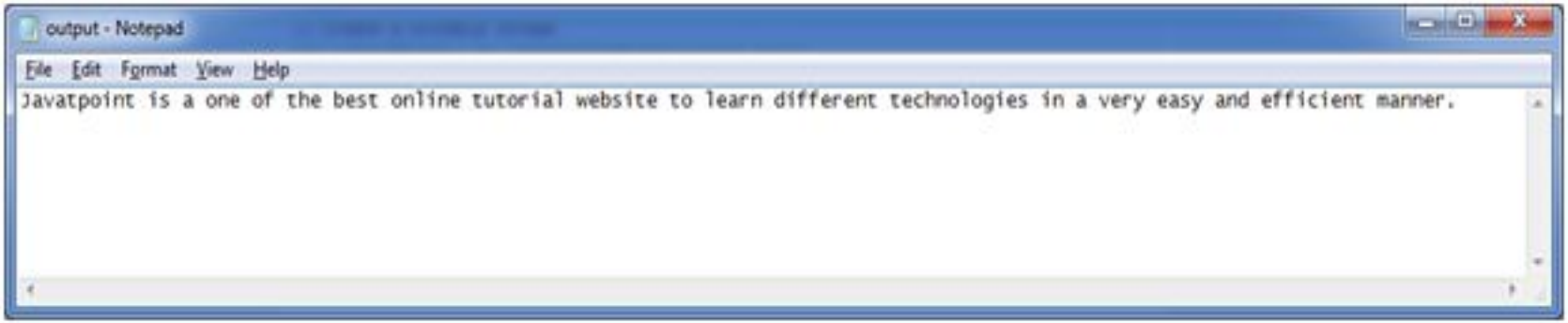
File: main.js

```
var fs = require("fs");
// Create a readable stream
var readerStream = fs.createReadStream('input.txt');
// Create a writable stream
var writerStream = fs.createWriteStream('output.txt');
// Pipe the read and write operations
// read input.txt and write data to output.txt
readerStream.pipe(writerStream);
console.log("Program Ended");
```

output


Node.js command prompt
Your environment has been set up for using Node.js 4.4.2 (x64) and npm.
C:\Users\javatpoint1>cd desktop
C:\Users\javatpoint1\Desktop>node main.js
Program Ended
C:\Users\javatpoint1\Desktop>_

Now, you can see that a text file named "output.txt" is created where you had saved ?main.js? file. In my case, it is on desktop.
Open the "output.txt" and you will see the following content.



Node.js Chaining Streams

Chaining stream is a mechanism of creating a chain of multiple stream operations by connecting output of one stream to another stream.

It is generally used with piping operation.

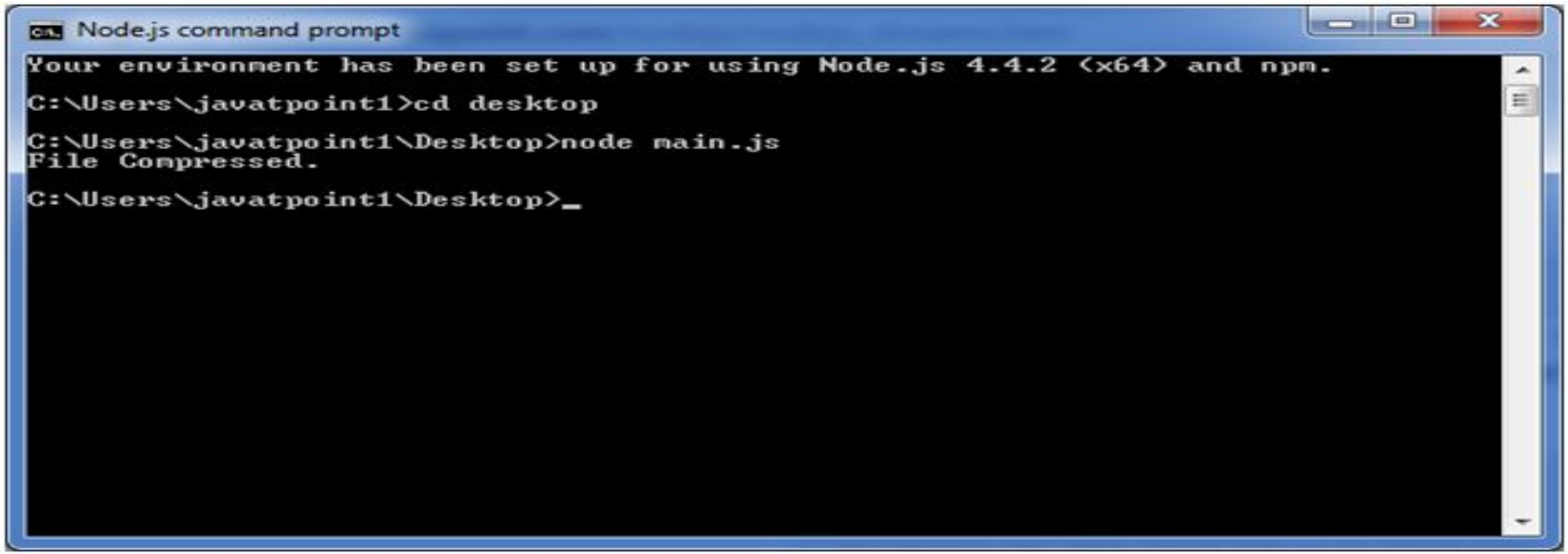
Let's take an example of piping and chaining to compress a file and then decompress the same file.

File: main.js

```
var fs = require("fs");
var zlib = require('zlib');
// Compress the file input.txt to input.txt.gz
fs.createReadStream('input.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('input.txt.gz'));
  console.log("File Compressed.");
```

Open the Node.js command prompt and run main.js

```
node main.js
```



rent file.

Now you will see that file "input.txt" is compressed and a new file is created named "input.txt.gz" in the current file.

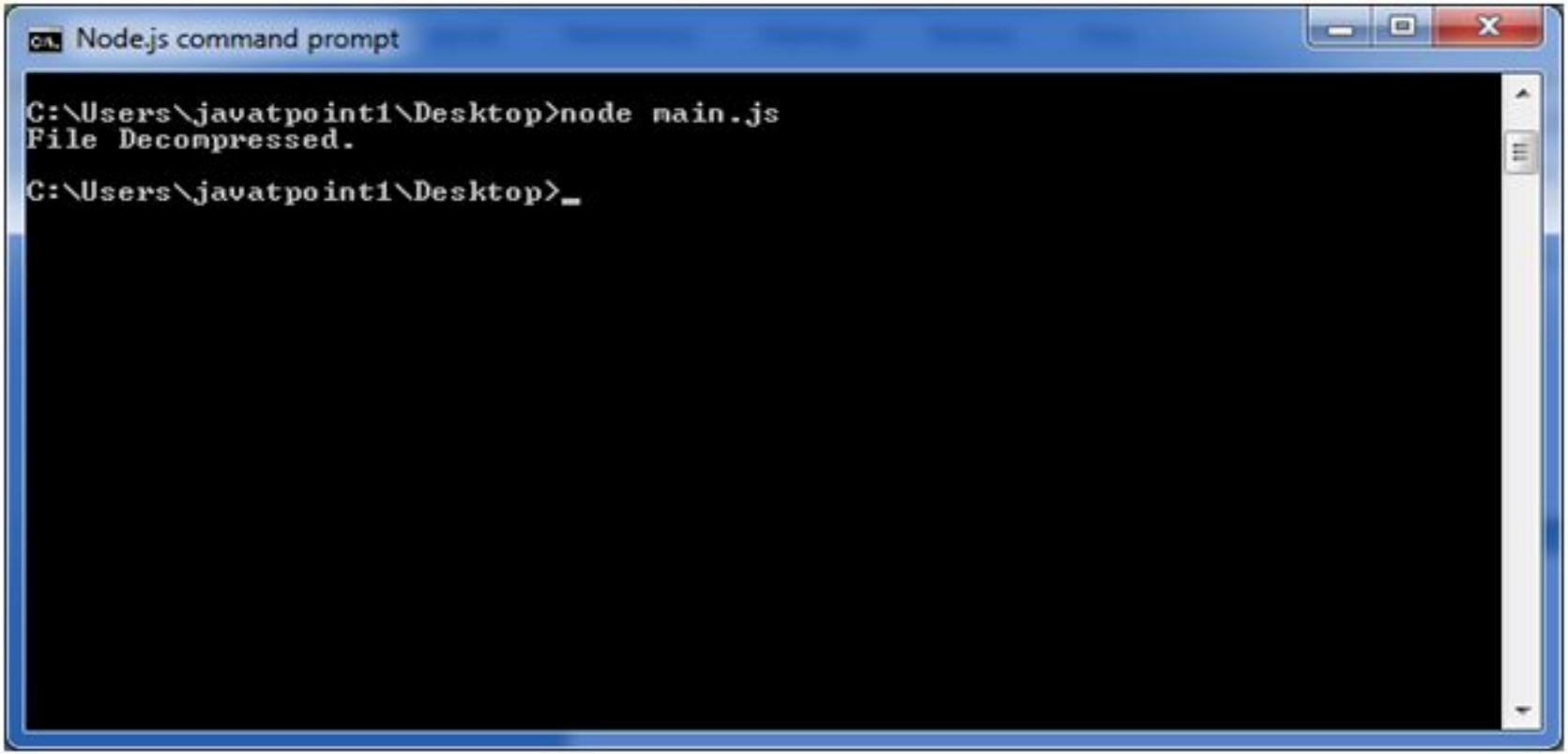
To Decompress the same file: put the following code in the js file "main.js"

File: main.js

```
var fs = require("fs");
var zlib = require('zlib');
// Decompress the file input.txt.gz to input.txt
fs.createReadStream('input.txt.gz')
  .pipe(zlib.createGunzip())
  .pipe(fs.createWriteStream('input.txt'));
  console.log("File Decompressed.");
```

Open the Node.js command prompt and run main.js

```
node main.js
```


Node.js command prompt
C:\Users\javatpoint1\Desktop>node main.js
File Decompressed.
C:\Users\javatpoint1\Desktop>

Thank you